

CLIPS PROGRAMMING

Weather Advisor Expert System

Complete Source Code Reference

File: `weather-advisor.clp`

Author: DennisCreatives Dennis Muthui

Language: CLIPS 6.4 • Paradigm: Rule-Based Expert System • Chaining: Forward

HOW TO RUN: 1. Load in CLIPS IDE: File → Load → weather-advisor.clp 2. Type (reset) then (run) at the CLIPS prompt 3. Answer the 5 weather input prompts 4. Read your personalised advisory report Online IDE: <https://www.clipsrules.net/CLIPSWebIDE/> VS Code extension: search 'CLIPS' by Rob McDaniel

SECTION 1 — DATA TEMPLATES (deftemplates)

Two deftemplates define the data structures used by the system. The weather-condition template holds all user input. The advice template stores each piece of generated advice.

```
;;; =====
;;; SECTION 1 — DATA TEMPLATES (deftemplates)
;;; =====

;;; --- Input fact: holds all observed weather measurements ---
(deftemplate weather-condition
  "Holds the current observed weather data entered by the user."
  (slot temperature (type FLOAT) (default 25.0)) ; Celsius
  (slot humidity (type FLOAT) (default 50.0)) ; percentage 0-100
  (slot sky (type SYMBOL) (default clear)) ; clear | cloudy | overcast
  (slot wind-speed (type FLOAT) (default 5.0)) ; km/h
  (slot rain-prob (type FLOAT) (default 0.1)) ; 0.0 to 1.0
)

;;; --- Output fact: stores one piece of advice to display ---
(deftemplate advice
  "Stores a single piece of advice generated by a rule."
  (slot category (type SYMBOL) (default general)) ; clothing | activity |
health | safety | general
  (slot message (type STRING) (default ""))
)

;;; =====
```

Why deftemplates? Unlike ordered facts, deftemplates give each field a named slot and an enforced type. This makes rules self-documenting and prevents type mismatches at runtime.

SECTION 2 — INPUT RULE (Salience 200 — fires first)

The get-weather-input rule has the highest salience (200) in the entire system, guaranteeing it fires before any analysis rules. The (not (weather-condition)) guard ensures it fires only once. It prompts the user for five inputs, then asserts the weather-condition fact into working memory.

```
(defrule get-weather-input
  "Collects all weather data from the user before any analysis rules fire."
  (declare (salience 200))
  (not (weather-condition)) ; guard: only fires if no condition exists yet
=>
  (printout t crlf)
  (printout t "===== " crlf)
  (printout t "    CLIPS WEATHER ADVISOR EXPERT SYSTEM    " crlf)
  (printout t "    DennisCreatives Data Solutions Ltd    " crlf)
  (printout t "===== " crlf crlf)

  (printout t "Please enter current weather data:" crlf)
  (printout t "-----" crlf)

  (printout t "Temperature (Celsius): ")
  (bind ?temp (read))

  (printout t "Humidity percentage (0-100): ")
  (bind ?hum (read))

  (printout t "Sky condition (clear / cloudy / overcast): ")
  (bind ?sky (read))

  (printout t "Wind speed (km/h): ")
  (bind ?wind (read))

  (printout t "Rain probability (0.0 to 1.0): ")
  (bind ?rain (read))

  (assert (weather-condition
    (temperature (float ?temp))
    (humidity (float ?hum))
    (sky ?sky)
    (wind-speed (float ?wind))
    (rain-prob (float ?rain))))

  (printout t crlf "Data recorded. Analysing..." crlf)
)
```


SECTION 3 — TEMPERATURE RULES (6 rules)

Six rules cover the full temperature spectrum from extreme cold to extreme heat. Rules are mutually exclusive by their (test) conditions. The extreme-heat and extreme-cold rules carry elevated salience values so they fire before their respective standard counterparts.

Rule Name	Condition	Salience / Output Category
extreme-cold	temp < 0°C	80 → clothing + health
cold-weather	0 ≤ temp < 15°C	0 → clothing
cool-weather	15 ≤ temp < 22°C	0 → clothing
warm-weather	22 ≤ temp < 30°C	0 → clothing
hot-weather	30 ≤ temp < 38°C	0 → clothing + health
extreme-heat	temp ≥ 38°C	90 → safety + health

```

;;; SECTION 3 — TEMPERATURE RULES
;;; =====

(defrule extreme-cold
  "Fires when temperature is below freezing (< 0°C).\"
  (declare (salience 80))
  (weather-condition (temperature ?t))
  (test (< ?t 0.0))
=>
  (assert (advice
    (category clothing)
    (message "EXTREME COLD: Wear thermal underlay, heavy coat, waterproof boots,
gloves, and a hat.)))
  (assert (advice
    (category health)
    (message "Risk of hypothermia and frostbite. Limit time outdoors. Keep
children and elderly indoors.)))
)

(defrule cold-weather
  "Fires when temperature is cold but above freezing (0-14°C).\"
  (weather-condition (temperature ?t))
  (test (and (>= ?t 0.0) (< ?t 15.0)))
=>
  (assert (advice
    (category clothing)
    (message "Cold weather: Wear a warm jacket, scarf, and closed shoes.)))
)

(defrule cool-weather

```

```
"Fires when temperature is cool and comfortable (15-21°C)."  
(weather-condition (temperature ?t))  
(test (and (>= ?t 15.0) (< ?t 22.0)))  
=>  
(assert (advice  
  (category clothing)  
  (message "Cool weather: A light jacket or sweater is recommended.")))  
)  
  
(defrule warm-weather  
  "Fires when temperature is warm and pleasant (22-29°C)."  
  (weather-condition (temperature ?t))  
  (test (and (>= ?t 22.0) (< ?t 30.0)))  
=>  
(assert (advice  
  (category clothing)  
  (message "Warm weather: Light casual clothes are comfortable today.")))  
)  
  
(defrule hot-weather  
  "Fires when temperature is hot (30-37°C)."  
  (weather-condition (temperature ?t))  
  (test (and (>= ?t 30.0) (< ?t 38.0)))  
=>  
(assert (advice  
  (category clothing)  
  (message "Hot weather: Wear light, breathable fabrics. Apply SPF 30+  
sunscreen.")))  
(assert (advice  
  (category health)  
  (message "Stay hydrated. Drink at least 2 litres of water. Avoid peak sun  
hours (10am-3pm).")))  
)  
  
(defrule extreme-heat  
  "Fires when temperature is dangerously high (>= 38°C). Higher salience = fires  
before hot-weather."  
(declare (salience 90))  
(weather-condition (temperature ?t))  
(test (>= ?t 38.0))
```

SECTION 4 — HUMIDITY RULES (4 rules)

Four rules classify humidity from very dry (<30%) to very high (≥80%). High and very-high humidity rules feed into the combined heat-index rule in Section 5 to catch the dangerous combination of heat and humidity.

```
(category safety)
(message "EXTREME HEAT WARNING: Risk of heat stroke! Stay indoors, use air
conditioning or fans.)))
(assert (advice
(category health)
(message "Drink water constantly. Check on elderly neighbours, young
children, and pets urgently.)))
)

;;; =====
;;; SECTION 4 — HUMIDITY RULES
;;; =====

(defrule very-dry
"Fires when humidity is very low (< 30%).\"
(weather-condition (humidity ?h))
(test (< ?h 30.0))
=>
(assert (advice
(category health)
(message "Very dry air: Use moisturiser. Risk of dry skin, chapped lips, and
irritated sinuses. Stay hydrated.)))
)

(defrule moderate-humidity
"Fires when humidity is in the comfortable range (30-59%).\"
(weather-condition (humidity ?h))
(test (and (>= ?h 30.0) (< ?h 60.0)))
=>
(assert (advice
(category general)
(message "Humidity is comfortable today. No special precautions needed.)))
)

(defrule high-humidity
"Fires when humidity is noticeably high (60-79%).\"
(weather-condition (humidity ?h))
(test (and (>= ?h 60.0) (< ?h 80.0)))
=>
(assert (advice
(category health)
```

```
        (message "High humidity: You will feel warmer than the actual temperature.  
Loose, breathable clothing recommended."))  
    )  
  
(defrule very-high-humidity  
    "Fires when humidity is oppressively high (>= 80%)."
```

SECTION 5 — COMBINED CONDITION RULES (3 rules)

These rules test multiple slots simultaneously. The heat-index-danger rule fires when BOTH temperature $\geq 30^{\circ}\text{C}$ AND humidity $\geq 70\%$ — a combination that makes conditions feel significantly hotter than the thermometer reads. The ideal-outdoor-day rule uses five simultaneous conditions with strict ranges for a fully clear, comfortable day.

```
=>
  (assert (advice
    (category health)
    (message "Very high humidity: Oppressive conditions. Strenuous outdoor
activity not advised. Risk of heat exhaustion.")))
)

;;; =====
;;; SECTION 5 — COMBINED CONDITION RULES (Heat Index, etc.)
;;; =====

(defrule heat-index-danger
  "Fires when both temperature AND humidity are high — feels much hotter than
thermometer reads."
  (declare (salience 60))
  (weather-condition (temperature ?t) (humidity ?h))
  (test (and (>= ?t 30.0) (>= ?h 70.0)))
=>
  (assert (advice
    (category safety)
    (message "HEAT INDEX ALERT: High temperature combined with high humidity
makes it feel significantly hotter. Risk of heat exhaustion is elevated.")))
)

(defrule ideal-outdoor-day
  "Fires only when ALL conditions are optimal for outdoor activity."
  (weather-condition
    (temperature ?t)
    (humidity ?h)
    (wind-speed ?w)
    (rain-prob ?r)
    (sky ?s))
  (test (and (>= ?t 20.0) (<= ?t 28.0)))
  (test (and (>= ?h 40.0) (<= ?h 65.0)))
  (test (< ?w 25.0))
  (test (< ?r 0.3))
  (test (eq ?s clear))
=>
  (assert (advice
    (category activity)
    (message "IDEAL OUTDOOR DAY: Perfect conditions for sports, hiking, cycling,
or a picnic. Enjoy the day!")))
```

```
)

(defrule good-for-jogging
  "Fires when conditions are suitable for a run."
  (weather-condition (temperature ?t) (wind-speed ?w) (rain-prob ?r))
  (test (and (>= ?t 12.0) (<= ?t 22.0)))
  (test (< ?w 20.0))
  (test (< ?r 0.4))
=>
  (assert (advice
    (category activity)
    (message "Good conditions for a morning jog, walk, or light outdoor
exercise.")))
)
```

SECTION 6 — RAIN PROBABILITY RULES (4 rules)

Rain probability is expressed as a float from 0.0 (no rain) to 1.0 (certain rain). The heavy-rain-warning rule carries salience 70 and asserts both a safety warning and an activity recommendation, since flooding is a safety concern, not just an inconvenience.

```
"Fires when rain or wind makes outdoor activity inadvisable."
(weather-condition (rain-prob ?r) (wind-speed ?w))
(test (or (>= ?r 0.7) (>= ?w 50.0)))
=>
(assert (advice
  (category activity)
  (message "Today is better suited for indoor activities: reading, studying,
cooking, gym, or cinema.")))
)

;;; =====
;;; SECTION 6 — RAIN PROBABILITY RULES
;;; =====

(defrule no-rain
  "Fires when rain probability is low and sky is clear."
  (weather-condition (rain-prob ?r) (sky ?s))
  (test (and (< ?r 0.3) (eq ?s clear))))
=>
(assert (advice
  (category activity)
  (message "No rain expected. Great day for outdoor activities!")))
)

(defrule possible-rain
  "Fires when there is a moderate chance of rain (30-59%)."
  (weather-condition (rain-prob ?r))
  (test (and (>= ?r 0.3) (< ?r 0.6))))
=>
(assert (advice
  (category activity)
  (message "Rain is possible later in the day. Consider carrying a light
umbrella or raincoat.")))
)

(defrule likely-rain
  "Fires when rain is likely (60-79%)."
  (weather-condition (rain-prob ?r))
  (test (and (>= ?r 0.6) (< ?r 0.8))))
=>
(assert (advice
  (category activity)
```

```
    (message "Rain is likely today. Bring a sturdy umbrella and waterproof
footwear."))
)

(defrule heavy-rain-warning
  "Fires when heavy rain is highly probable (>= 80%). Safety priority."
  (declare (salience 70))
  (weather-condition (rain-prob ?r))
  (test (>= ?r 0.8))
```

SECTION 7 – WIND SPEED RULES (5 rules)

Five rules span from calm (< 10 km/h) through to dangerous storm-force winds (≥ 60 km/h). The dangerous-storm-wind rule carries the highest salience in the system (100) so it fires before all other wind or temperature rules when storm conditions are detected.

```
(category safety)
(message "HEAVY RAIN WARNING: Very high flood risk. Avoid low-lying areas,
open drains, and river banks.)))
(assert (advice
(category activity)
(message "Postpone all outdoor plans. Stay indoors if possible. Do not
attempt to drive through flooded roads.)))
)

;;; =====
;;; SECTION 7 – WIND SPEED RULES
;;; =====

(defrule calm-wind
  "Fires when wind is calm (< 10 km/h)."
  (weather-condition (wind-speed ?w))
  (test (< ?w 10.0))
=>
  (assert (advice
    (category activity)
    (message "Wind is calm. Perfect conditions for cycling, flying a kite, or
outdoor dining.)))
  )

(defrule light-breeze
  "Fires when there is a light, pleasant breeze (10-24 km/h)."
  (weather-condition (wind-speed ?w))
  (test (and (>= ?w 10.0) (< ?w 25.0)))
=>
  (assert (advice
    (category activity)
    (message "Light breeze. Pleasant conditions for most outdoor activities.)))
  )

(defrule moderate-wind
  "Fires when wind is moderately strong (25-39 km/h)."
  (weather-condition (wind-speed ?w))
  (test (and (>= ?w 25.0) (< ?w 40.0)))
=>
  (assert (advice
    (category activity)
```

```
    (message "Moderate wind. Secure loose outdoor items. Driving may feel
slightly affected on exposed roads."))
)

(defrule strong-wind-advisory
  "Fires when wind is strong and potentially hazardous (40-59 km/h).\"
  (declare (salience 80))
  (weather-condition (wind-speed ?w))
  (test (and (>= ?w 40.0) (< ?w 60.0)))
=>
  (assert (advice
    (category safety)
    (message "STRONG WIND ADVISORY: Avoid standing under tall trees or unsecured
structures. Motorcyclists exercise caution.")))
)
```

SECTION 8 — OUTPUT RULES (6 rules — negative salience)

Output rules all have negative salience values, guaranteeing they fire after every analysis rule has completed. Safety advice prints first (salience -150), followed by health, clothing, activity, and general. The print-report-footer rule at salience -999 is the very last thing that fires. Each output rule uses a (not (printed-X ?msg)) guard to prevent duplicate printing.

```
(declare (salience 100))
(weather-condition (wind-speed ?w))
(test (>= ?w 60.0))
=>
  (assert (advice
    (category safety)
    (message "DANGEROUS WIND: Storm-force conditions! Stay indoors. Secure all
loose objects. Power outages possible.")))
)

;;; =====
;;; SECTION 8 — OUTPUT RULES (Salience negative — fire last)
;;; =====

(defrule print-report-header
  "Prints the advisory report header. Fires after all analysis rules."
  (declare (salience -100))
  (not (report-started)) ; guard: fires only once
=>
  (printout t crlf)
  (printout t "===== " crlf)
  (printout t "          WEATHER ADVISORY REPORT          " crlf)
  (printout t "===== " crlf crlf)
  (assert (report-started))
)

(defrule print-safety-advice
  "Prints all SAFETY category advice first."
  (declare (salience -150))
  (report-started)
  (advice (category safety) (message ?msg))
  (not (printed-safety ?msg))
=>
  (printout t "[SAFETY]  " ?msg crlf crlf)
  (assert (printed-safety ?msg))
)

(defrule print-health-advice
  "Prints all HEALTH category advice."
  (declare (salience -160))
  (report-started)
```

```
(advice (category health) (message ?msg))
(not (printed-health ?msg))
=>
(printout t "[HEALTH] " ?msg crlf crlf)
(assert (printed-health ?msg))
)

(defrule print-clothing-advice
  "Prints all CLOTHING category advice."
  (declare (salience -170))
  (report-started)
  (advice (category clothing) (message ?msg))
  (not (printed-clothing ?msg))
=>
  (printout t "[CLOTHING] " ?msg crlf crlf)
  (assert (printed-clothing ?msg))
)

(defrule print-activity-advice
  "Prints all ACTIVITY category advice."
  (declare (salience -180))
  (report-started)
  (advice (category activity) (message ?msg))
  (not (printed-activity ?msg))
=>
  (printout t "[ACTIVITY] " ?msg crlf crlf)
  (assert (printed-activity ?msg))
)

(defrule print-general-advice
  "Prints all GENERAL category advice."
  (declare (salience -190))
```

SECTION 9 — TEST SCENARIOS (4 commented deffacts)

Four ready-made test scenarios are included as commented-out deffacts. To use a scenario: (1) comment out the get-weather-input rule in Section 2, (2) uncomment the desired deffacts block below, (3) run (reset) then (run). This is ideal for classroom demonstrations without needing manual input.

```
(not (printed-general ?msg))
=>
  (printout t "[GENERAL] " ?msg crlf crlf)
  (assert (printed-general ?msg))
)

(defrule print-report-footer
  "Prints the closing footer. Fires absolutely last."
  (declare (salience -999))
  (report-started)
=>
  (printout t "=====" crlf)
  (printout t "    End of Advisory. Stay safe!" crlf)
  (printout t "    DennisCreatives Data Solutions Ltd" crlf)
  (printout t "=====" crlf crlf)
)

;;; =====
;;; SECTION 9 — OPTIONAL: MANUAL TEST (comment out for interactive)
;;;
;;; To test without user input, comment out the get-weather-input
;;; rule above and uncomment ONE of the test scenarios below.
;;; Then run: (reset) (run)
;;; =====

;;; --- TEST SCENARIO A: Hot & Humid Nairobi Day ---
;; (deffacts test-scenario-A
;;   "Nairobi hot rainy afternoon — 34°C, 75% humidity, cloudy, 15 km/h wind,
;;   0.65 rain."
;;   (weather-condition
;;     (temperature 34.0)
;;     (humidity 75.0)
;;     (sky cloudy)
;;     (wind-speed 15.0)
;;     (rain-prob 0.65))
;; )

;;; --- TEST SCENARIO B: Perfect Outdoor Day ---
;; (deffacts test-scenario-B
;;   "Ideal outdoor day — 24°C, 55% humidity, clear sky, 10 km/h wind, 0.1 rain."
;;   (weather-condition
;;     (temperature 24.0)
```

```
;;      (humidity      55.0)
;;      (sky           clear)
;;      (wind-speed    10.0)
;;      (rain-prob     0.1))
;; )

;;; --- TEST SCENARIO C: Extreme Storm ---
;; (defacts test-scenario-C
;;   "Storm conditions - 20°C, 90% humidity, overcast, 70 km/h wind, 0.95 rain."
;;   (weather-condition
;;     (temperature 20.0)
;;     (humidity    90.0)
;;     (sky         overcast)
;;     (wind-speed  70.0)
;;     (rain-prob   0.95))
;;   )
```

Scenario	Temperature	Sky / Wind	Expected Outputs
A — Nairobi Hot	34°C, 75% RH	Cloudy, 15 km/h, 65% rain	Heat index alert, heavy clothing, rain warning
B — Ideal Day	24°C, 55% RH	Clear, 10 km/h, 10% rain	Ideal outdoor day, no umbrella needed
C — Storm	20°C, 90% RH	Overcast, 70 km/h, 95% rain	Storm wind danger, heavy rain, flood risk
D — Cold Morning	8°C, 40% RH	Clear, 5 km/h, 5% rain	Cold clothing, calm wind, no rain

End of Document — weather-advisor.clp
DennisCreatives Data Solutions Ltd • Kahawa Sukari, Nairobi, Kenya